

JavaScript — Complete Course Notes

Practical notes with examples for every core topic. Run in browser DevTools, Node, or your editor.

1 — Introduction to JavaScript

JavaScript (JS) is a high-level, dynamic language used to make web pages interactive. It runs in browsers and on servers (Node.js).

- **Client-side:** DOM, events, forms, animations
- **Server-side:** APIs, tooling, automation (Node.js)
- **ECMAScript:** The official language specification browsers implement

Add JS to HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>JS demo</title>
  </head>
  <body>
    <p id="msg">Hello</p>
    <script src="app.js"></script>
    <!-- or inline -->
    <script>
      console.log("inline script");
    </script>
  </body>
</html>
```

First program

```
console.log("Hello, JavaScript!");
```

Where code runs

- **Browser:** `window`, `document`, DOM APIs
- **Node:** `fs`, `path`, no `document` unless you use a browser automation tool

2 — Variables: `var`, `let`, `const`

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function	Block	Block
Reassign	Yes	Yes	No
Redeclare same scope	Yes	No	No
Must initialize	No	No	Yes
Default choice today	Avoid	When reassign needed	Prefer

```
const PI = 3.14;
let count = 0;
count += 1;

var oldStyle = 1; // function-scoped, avoid in new code
```

`const` and objects

```
const user = { name: "Ada" };
user.name = "Grace"; // OK – mutate object
// user = {};        // Error – cannot reassign binding
```

3 – Data types (primitives)

JS is **dynamically typed** – the variable has no fixed type; the **value** does.

Seven primitive types

Type	Example	Notes
<code>number</code>	<code>42</code> , <code>3.14</code> , <code>NaN</code>	All numbers are IEEE floats
<code>string</code>	<code>"hi"</code> , <code>`hi`</code>	Immutable text
<code>boolean</code>	<code>true</code> , <code>false</code>	
<code>undefined</code>	<code>undefined</code>	Unassigned / missing
<code>null</code>	<code>null</code>	Intentional empty
<code>symbol</code>	<code>Symbol("id")</code>	Unique keys
<code>bigint</code>	<code>9007199254740993n</code>	Large integers

Reference types (not primitive)

Objects, arrays, functions – stored by **reference**.

```
const a = 5;
const b = a;
b = 10;
console.log(a); // 5 (copy value)

const o1 = { x: 1 };
const o2 = o1;
o2.x = 2;
console.log(o1.x); // 2 (same object)
```

4 — typeof and type checking

```
console.log(typeof 42);           // "number"
console.log(typeof "hi");         // "string"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof null);         // "object" (historical quirk)
console.log(typeof Symbol());     // "symbol"
console.log(typeof 1n);           // "bigint"
console.log(typeof {});           // "object"
console.log(typeof []);           // "object"
console.log(typeof function(){}); // "function"
```

Better checks

```
Array.isArray([1, 2]);           // true
Number.isNaN(NaN);               // true (prefer over global isNaN)
Number.isFinite(100);            // true
```

5 — Operators: arithmetic and comparison

Arithmetic

```
console.log(10 + 3);             // 13
console.log(10 - 3);             // 7
console.log(10 * 3);             // 30
console.log(10 / 3);             // 3.333...
console.log(10 % 3);             // 1 (remainder)
console.log(2 ** 8);             // 256 (exponent)
console.log(0.1 + 0.2);          // 0.30000000000000004 (float quirk)
```

String + vs number +

```
console.log("5" + 1);           // "51" (string concat)
console.log("5" - 1);           // 4 (coerces to number)
console.log(Number("42"));       // 42
```

Assignment operators

```
let x = 10;
x += 2; // x = 12
x -= 3; // x = 9
x *= 2; // x = 18
x /= 3; // x = 6
```

Increment / decrement

```
let n = 5;
console.log(n++); // 5 then n is 6
console.log(++n); // 7
```

Comparison — prefer `===` and `!==`

```
console.log(5 === 5);    // true
console.log(5 === "5");  // false (no coercion)
console.log(5 == "5");   // true (coercion – avoid)
console.log(5 !== "5");  // true
console.log(10 > 3);     // true
console.log(null == undefined); // true
console.log(null === undefined); // false
```

6 – Logical operators

Operator	Meaning
<code>&&</code>	AND – short-circuit
<code> </code>	OR – short-circuit
<code>!</code>	NOT

```
const age = 20;
const hasTicket = true;
const canEnter = age >= 18 && hasTicket;

console.log(false || "default"); // "default"
console.log(0 && "skipped");      // 0

// Nullish coalescing ?? – only null/undefined
const port = 0 ?? 3000; // 0
const p2 = null ?? 3000; // 3000
```

7 – Conditionals: `if`, `else`, `switch`

`if` / `else if` / `else`

```
const score = 72;
if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else {
  console.log("Below B");
}
```

`switch`

```
function dayName(n) {
  switch (n) {
    case 1:
      return "Mon";
    case 2:
      return "Tue";
    default:
      return "Unknown";
  }
}
```

Use `break` between cases unless you intentionally fall through.

8 — Truthy and falsy

Falsy: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`

Truthy: everything else (`"0"`, `[]`, `{}`, `"false"`)

```
if ("" ) {
  console.log("never");
} else {
  console.log("empty string is falsy");
}

if ("0") {
  console.log("non-empty string is truthy");
}
```

9 — Loops: `for`

```
for (let i = 0; i < 5; i++) {
  console.log(i); // 0 1 2 3 4
}

const items = ["a", "b", "c"];
for (let i = 0; i < items.length; i++) {
  console.log(items[i]);
}
```

10 — Loops: `while` and `do...while`

```
let n = 3;
while (n > 0) {
  console.log(n);
  n--;
}

let m = 0;
do {
  console.log("runs at least once");
  m++;
} while (m < 0);
```

11 — Loops: `forEach`

Array method — runs a callback for each element. **Cannot** `break` early (use `for` / `for...of` instead).

```
[10, 20, 30].forEach((value, index) => {
  console.log(index, value);
});
```

Returns `undefined` — use for **side effects**, not building a new array.

12 — Loops: `for...of` and `for...in`

`for...of` — values of iterables (arrays, strings, Map, Set)

```
for (const ch of "hi") {
  console.log(ch); // h, i
}

for (const n of [1, 2, 3]) {
  console.log(n);
}
```

`for...in` — enumerable keys (objects; on arrays gives index strings)

```
const user = { name: "Ada", role: "dev" };
for (const key in user) {
  console.log(key, user[key]);
}
```

Prefer `for...of` for arrays.

13 — `break` and `continue`

```
for (let i = 0; i < 10; i++) {
  if (i === 3) continue; // skip this iteration
  if (i === 7) break;    // exit loop
  console.log(i);
}
```

Works in `for`, `while`, `do...while`. Not in `forEach` callbacks.

14 — Functions: declaration and expression

Function declaration (hoisted)

```
console.log(add(2, 3)); // 5 — can call before line in file

function add(a, b) {
  return a + b;
}
```

Function expression (not hoisted like declaration)

```
const multiply = function (a, b) {
  return a * b;
};
```

Named function expression

```
const factorial = function fact(n) {
  if (n <= 1) return 1;
  return n * fact(n - 1);
};
```

15 — Function parameters and return

```
function greet(name = "Guest") {  
  return `Hello, ${name}`;  
}  
  
function sum(...nums) {  
  return nums.reduce((a, b) => a + b, 0);  
}  
  
console.log(sum(1, 2, 3)); // 6  
  
function noop() {  
  // no return → undefined  
}
```

16 — Arrow functions

```
const double = (x) => x * 2;  
  
const triple = (x) => {  
  return x * 3;  
};  
  
const team = {  
  name: "Apex",  
  log() {  
    this.members.forEach((m) => {  
      console.log(this.name, m); // `this` from log()  
    });  
  },  
  members: ["li", "jan"],  
};
```

- No own `this` (lexical)
- No `arguments` object — use rest `...args`
- Cannot use `new` with arrow functions

17 — Scope: global and function

Global scope: top-level bindings visible everywhere (in browser, `var` at top level attaches to `window`).

Function scope: variables declared with `var` or parameters exist inside that function only.

```
const globalVar = "I am global";  
  
function outer() {  
  const fnScoped = "inside outer";  
  function inner() {  
    console.log(globalVar, fnScoped);  
  }  
  inner();  
}
```

18 — Scope: block scope

`let` and `const` are **block-scoped** — limited to nearest `{ }`.

```
if (true) {  
  let x = 10;  
  const y = 20;  
}  
// console.log(x); // ReferenceError
```

19 — Hoisting

Declarations are moved in concept; behavior differs:

```
console.log(a); // undefined (var hoisted + initialized undefined)  
var a = 1;  
  
// console.log(b); // ReferenceError (let in TDZ)  
let b = 2;
```

Temporal Dead Zone (TDZ): `let` / `const` exist but cannot be read until their declaration line runs.

Function declarations are fully hoisted:

```
hi(); // works  
function hi() {  
  console.log("hi");  
}
```

20 — Closures (intro)

A **closure** is when an inner function remembers variables from its outer scope after the outer function has finished.

```
function makeCounter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter = makeCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

Use cases: private state, factories, callbacks.

21 — Arrays: create and access

```
const fruits = ["apple", "banana", "cherry"];  
console.log(fruits[0]); // "apple"  
console.log(fruits.at(-1)); // "cherry" (last)  
  
const fromRange = Array.from({ length: 5 }, (_, i) => i); // [0,1,2,3,4]
```

22 — Arrays: `length` and mutating

```
const a = [1, 2, 3];
console.log(a.length); // 3

a.length = 5; // grows with empty slots
a[10] = "x"; // sparse array
```

Mutating methods change the original array: `push`, `pop`, `splice`, `sort`, etc.

23 — Array: `push`, `pop`, `shift`, `unshift`

```
const a = [2, 3];
a.push(4); // [2,3,4] - add end
a.pop(); // removes 4
a.unshift(1); // [1,2,3] - add start
a.shift(); // removes 1
```

`shift` / `unshift` are slower on large arrays (re-index everything).

24 — Array: `slice` and `splice`

`slice` — non-mutating copy of portion

```
const a = [10, 20, 30, 40];
const part = a.slice(1, 3); // [20, 30], a unchanged
```

`splice` — mutating insert/delete

```
const b = [10, 20, 30, 40];
const removed = b.splice(1, 2, 99); // removed [20,30], b = [10, 99, 40]
```

25 — Array: `map`

Returns a **new array** of the same length.

```
const nums = [1, 2, 3];
const doubled = nums.map((n) => n * 2);
console.log(doubled); // [2, 4, 6]
```

26 — Array: `filter`

Keeps elements where callback returns truthy.

```
const evens = [1, 2, 3, 4].filter((n) => n % 2 === 0);
console.log(evens); // [2, 4]
```

27 — Array: `reduce`

Folds array to one value (or object).

```
const total = [10, 20, 30].reduce((sum, n) => sum + n, 0);
console.log(total); // 60
```

28 — Array: `find` and `findIndex`

```
const users = [{ id: 1, name: "A" }, { id: 2, name: "B" }];
const u = users.find((x) => x.id === 2);
const i = users.findIndex((x) => x.id === 2);
```

Returns `undefined` / `-1` if not found.

29 — Array: `some` and `every`

```
const nums = [2, 4, 6];
console.log(nums.some((n) => n % 2 !== 0)); // false
console.log(nums.every((n) => n % 2 === 0)); // true
```

30 — Array: `sort` and `reverse`

Default `sort()` compares as strings — use a comparator for numbers.

```
const a = [10, 2, 1];
a.sort((x, y) => x - y);
console.log(a); // [1, 2, 10]

const b = [1, 2, 3];
b.reverse(); // [3, 2, 1] — mutating
```

31 — Array: `concat` and spread

```
const a = [1, 2];
const b = [3, 4];
console.log(a.concat(b)); // [1,2,3,4]
console.log([...a, ...b]); // [1,2,3,4]
```

Spread creates a shallow copy.

32 — Array: `flat` and `flatMap`

```
const nested = [1, [2, 3], [4, [5]]];
console.log(nested.flat(1)); // [1, 2, 3, 4, [5]]

const lines = ["a b", "c d"];
console.log(lines.flatMap((s) => s.split(" "))); // ["a","b","c","d"]
```

33 — Array: `from` and `isArray`

```
console.log(Array.from("abc")); // ["a","b","c"]
console.log(Array.from({ length: 3 }, (_, i) => i * 2)); // [0,2,4]
console.log(Array.isArray([])); // true
console.log(Array.isArray({})); // false
```

34 — Objects: create and properties

```
const person = {
  name: "Ada",
  age: 36,
  greet() {
    return `Hi, ${this.name}`;
  },
};

console.log(person.greet());
```

35 — Objects: access and modify

```
const o = { a: 1, "full-name": "X" };
console.log(o.a);
console.log(o["full-name"]);

o.b = 2;
delete o.a;
```

36 — Objects: `keys`, `values`, `entries`

```
const scores = { math: 90, eng: 85 };
console.log(Object.keys(scores)); // ["math","eng"]
console.log(Object.values(scores)); // [90, 85]
console.log(Object.entries(scores)); // [["math",90],["eng",85]]

for (const [subject, score] of Object.entries(scores)) {
  console.log(subject, score);
}
```

37 — Object: spread and `Object.assign`

```
const defaults = { theme: "light", lang: "en" };
const userPrefs = { lang: "hi" };
const settings = { ...defaults, ...userPrefs };
// { theme: "light", lang: "hi" }

const target = {};
Object.assign(target, defaults, userPrefs);
```

Shallow merge — nested objects are still shared by reference.

38 — Object destructuring

```
const user = { name: "Ada", age: 36 };
const { name, age } = user;

const { city = "Unknown" } = user;

const nums = [1, 2, 3];
const [first, , third] = nums;
```

39 — Strings: methods

```
const s = " Hello, World ";
console.log(s.length);
console.log(s.trim());
console.log(s.toUpperCase());
console.log(s.toLowerCase());
console.log(s.includes("World"));
console.log(s.startsWith(" He"));
console.log(s.replace("World", "JS"));
console.log(s.split(", ")); // [" Hello", "World "]
console.log("a".repeat(3)); // "aaa"
```

40 — Template literals

```
const name = "Ada";
const year = 2026;
const msg = `Hello ${name}, year ${year}`;

const multiline = `
  line 1
  line 2
`;
```

Tagged templates (advanced): `fn`value`` .

41 — Number and Math

```
console.log(Number.parseInt("42px", 10)); // 42
console.log(Number.isInteger(3.0));        // true

console.log(Math.round(4.6)); // 5
console.log(Math.floor(4.9)); // 4
console.log(Math.ceil(4.1));  // 5
console.log(Math.max(1, 5, 3)); // 5
console.log(Math.random());    // 0 <= n < 1
```

42 — Date basics

```
const now = new Date();
console.log(now.toISOString());

const birthday = new Date(1990, 0, 15); // month 0 = January
console.log(birthday.getFullYear());
```

Prefer ISO strings for APIs; be careful with time zones.

43 — `JSON.stringify` and `JSON.parse`

```
const obj = { name: "Ada", skills: ["JS", "math"] };
const json = JSON.stringify(obj);
console.log(json);

const copy = JSON.parse(json);
console.log(copy.name);

// JSON does not support: undefined, functions, Symbol, Date (becomes string)
```

44 — Introduction to the DOM

The **DOM (Document Object Model)** is the browser's tree representation of HTML. JavaScript can read and change it.

```
document
├── html
│   ├── head
│   └── body
│       └── elements (nodes)
```

Node types you will use

- **Element nodes:** `<div>` , `<p>` , `<button>`
- **Text nodes:** text inside elements
- **Document:** `document` root

```
<div id="app">
  <p class="msg">Hello</p>
  <button id="btn">Click</button>
</div>
```

```
// document is the entry point
console.log(document.title);
document.title = "My Page";
```

Create and insert elements

```
const div = document.createElement("div");
div.textContent = "New block";
div.className = "card";
document.body.appendChild(div);
```

45 – Selecting elements

```
document.getElementById("app");
document.querySelector(".card"); // first match
document.querySelectorAll("li"); // NodeList (all matches)
document.getElementsByClassName("x");
document.getElementsByTagName("p");
```

Prefer `querySelector` / `querySelectorAll` with CSS selectors.

46 – Modifying content and attributes

```
const el = document.querySelector("#title");
el.textContent = "Plain text only";
el.innerHTML = "<em>HTML</em>"; // parses HTML – careful with user input

el.setAttribute("data-id", "42");
el.getAttribute("data-id");
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("open");
el.style.color = "crimson";
```

47 – Events: `addEventListener`

```
const btn = document.querySelector("#btn");

function handleClick(event) {
  console.log("clicked", event.type);
}

btn.addEventListener("click", handleClick);
btn.removeEventListener("click", handleClick);
```

Common events: `click` , `input` , `change` , `submit` , `keydown` , `load` .

48 – Event object and propagation

```
document.querySelector("#parent").addEventListener("click", (e) => {
  console.log("target:", e.target); // element clicked
  console.log("currentTarget:", e.currentTarget); // element with listener
  e.preventDefault(); // e.g. stop link navigation / form submit
  e.stopPropagation(); // stop bubbling to parents
});
```

Phases: `capture` → `target` → `bubble`

```
// Capture phase (third argument true)
parent.addEventListener("click", () => console.log("capture"), true);
child.addEventListener("click", () => console.log("bubble"));
```

Event delegation – one listener for many dynamic children:

```
document.querySelector("#list").addEventListener("click", (e) => {
  if (e.target.matches("button.delete")) {
    e.target.closest("li)?.remove();
  }
});
```

Keyboard snippet

```
input.addEventListener("keydown", (e) => {
  if (e.key === "Enter") console.log("submit");
});
```

49 — Forms: get value and submit

```
<form id="signup">
  <input name="email" type="email" />
  <button type="submit">Sign up</button>
</form>
```

```
const form = document.querySelector("#signup");

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const email = form.elements.email.value;
  console.log(email);

  const data = Object.fromEntries(new FormData(form));
  console.log(data);
});
```

50 — `setTimeout` and `setInterval`

```
const id = setTimeout(() => {
  console.log("once after delay");
}, 1000);

clearTimeout(id);

const intervalId = setInterval(() => {
  console.log("every second");
}, 1000);

clearInterval(intervalId);
```

Timers are **macrotasks** — they run after current synchronous code finishes.

51 — Error handling: `try` / `catch`

```
function parseJSON(text) {
  try {
    return JSON.parse(text);
  } catch (err) {
    console.error("Invalid JSON:", err.message);
    return null;
  } finally {
    console.log("parse attempt finished");
  }
}
```

Throw your own: `throw new Error("something went wrong");`

52 — Strict mode

```
"use strict";

// x = 10; // ReferenceError in strict mode
let x = 10;
```

- ES modules are strict by default
 - Prevents accidental globals, some unsafe syntax
-

53 — Debugging: `console`

```
console.log("info", { a: 1 });
console.warn("warning");
console.error("error");
console.table([{ id: 1 }, { id: 2 }]);
console.time("fetch");
// ... work ...
console.timeEnd("fetch");

const data = { user: { name: "Ada" } };
console.log(data.user?.name);
debugger; // pauses if DevTools open
```

54 — Code style and best practices

1. **Prefer `const`** , then `let` ; avoid `var`
2. **Use `===`** unless you have a deliberate coercion reason
3. **Name clearly:** `isActive` , `fetchUser` , `totalPrice`
4. **Small functions** — one job each
5. **Avoid implicit globals** — always declare variables
6. **Immutability when possible:** `map` / `filter` / `slice` instead of mutating shared state
7. **Handle errors** at boundaries (parse, fetch, user input)
8. **Sanitize** before `innerHTML` with untrusted data
9. **Use `===` / optional chaining `?.` / nullish `??`** for safer defaults
10. **Format consistently** (Prettier, ESLint)

Quick revision checklist

- ☐ Primitives vs references
- ☐ Truthy/falsy and `===`
- ☐ Loops vs array methods (`map` , `filter` , `reduce`)
- ☐ Scope, hoisting, closures
- ☐ DOM select → modify → listen
- ☐ JSON, timers, try/catch

End of course notes